

Mathematical Foundations of Distributed Data Systems

Companion to Session 1 | Big Data Engineering Exercise Sheet (without solutions)

Probability, Combinatorics & Reliability Analysis
for Replication, Partitioning, and the CAP Theorem

Document Contents

- Probability model for hash partitioning and load balancing.
- Reliability analysis of replicated systems.
- Quorum arithmetic and the consistency–availability inequality.
- Throughput, latency, and Little’s Law for streaming pipelines.
- **Two exercises (statements only)** to be solved by the student.

Contents

1	Introduction	2
2	Hash Partitioning: A Probabilistic View	2
2.1	The balls-and-bins model	2
2.2	Expected load and skew	2
2.3	Maximum load (the “hottest partition”)	3
3	Replication and Reliability	3
3.1	A simple Bernoulli model	3
3.2	Why this matters	3
3.3	Tolerated failures	3
4	Quorum Systems: $R + W > N$	4
4.1	The intersection argument	4
4.2	Tuning the trade-off	4
4.3	Connection to the CAP theorem	5
5	Throughput, Latency, and Little’s Law	5
5.1	Why Kafka measures both	5
5.2	Application to Kafka	5
6	Exercises	6
7	Summary	8
7.1	Going further	8

1. Introduction

In Session 1, we introduced distributed systems, the CAP theorem, partitioning, replication, and Apache Kafka. Behind every one of these design choices lies a small piece of *mathematics*: a probability model, a counting argument, or a reliability calculation. The goal of this companion document is to make those mathematical foundations explicit, so that you can reason about distributed systems quantitatively rather than only descriptively.

We focus on four building blocks:

1. **Hash partitioning** as a uniform random allocation $\rightarrow$ load balancing and the balls-and-bins model.
2. **Replication** as a Bernoulli reliability model $\rightarrow$ probability of data loss.
3. **Quorum systems** as set-intersection arguments $\rightarrow$ the inequality $R + W > N$.
4. **Throughput and latency** via Little's Law $\rightarrow$ capacity planning for Kafka pipelines.

2. Hash Partitioning: A Probabilistic View

2.1 The balls-and-bins model

Recall from Session 1 that Kafka uses **hash partitioning** to distribute records across partitions. Given a record key k , the producer computes $\text{partition}(k) = h(k) \bmod P$, where h is a hash function and P is the number of partitions.

If we model a good hash function as a uniformly random map, then assigning n keys to P partitions becomes the classical *balls-and-bins* problem.

Definition – Balls-and-Bins Model

We throw n independent balls into P bins, each ball landing in any given bin with probability $1/P$. Let X_i denote the number of balls in bin i . Then for each $i \in \{1, \dots, P\}$:

$$X_i \sim \mathcal{B}\left(n, \frac{1}{P}\right), \quad \mathbb{E}[X_i] = \frac{n}{P}, \quad \text{Var}(X_i) = \frac{n}{P} \left(1 - \frac{1}{P}\right).$$

2.2 Expected load and skew

If the keys are uniform, every partition receives the same expected load n/P . The variance is small relative to the mean as $n \rightarrow \infty$, so the *relative* imbalance vanishes. This is precisely why hash partitioning gives good load balancing for large workloads.

Coefficient of Variation

$$\frac{\sqrt{\text{Var}(X_i)}}{\mathbb{E}[X_i]} = \sqrt{\frac{P-1}{n}} \xrightarrow{n \rightarrow \infty} 0.$$

The relative load imbalance shrinks like $1/\sqrt{n}$.

2.3 Maximum load (the “hottest partition”)

A more refined question: what is the load of the *busiest* partition? A classical result states that, when $n = P$ (one ball per bin on average), the maximum bin load is $\Theta\left(\frac{\log P}{\log \log P}\right)$ with high probability. For Kafka this means: even with perfect hashing, you should expect some partitions to receive a few times more traffic than average if the number of keys is small.

i Important Remark

A skewed key distribution—e.g. 80% of events sharing the same `user_id`—defeats hashing entirely. The mathematical assumption of uniform keys is broken, and one partition becomes a bottleneck. This is the origin of “hot partitions” in production Kafka.

3. Replication and Reliability

3.1 A simple Bernoulli model

Let each broker fail independently during a given time window with probability p . A partition replicated on r brokers (replication factor r) is lost only if *all* r replicas fail simultaneously.

X¹ Theorem – Probability of Data Loss

Under the assumption of independent broker failures with failure probability p and replication factor r :

$$\mathbb{P}(\text{partition lost}) = p^r.$$

The probability that the partition survives is $1 - p^r$.

3.2 Why this matters

Even modest values of r provide enormous reliability gains because the failure probability decreases *exponentially* in r .

p (per broker)	$r = 1$	$r = 2$	$r = 3$
10^{-2}	10^{-2}	10^{-4}	10^{-6}
10^{-3}	10^{-3}	10^{-6}	10^{-9}
10^{-4}	10^{-4}	10^{-8}	10^{-12}

3.3 Tolerated failures

A replication factor of r tolerates up to $r - 1$ *simultaneous* broker failures without data loss. This is a worst-case combinatorial guarantee, independent of the failure probability p .

Fault Tolerance Bound

With replication factor r :

$$\text{maximum tolerated simultaneous failures} = r - 1.$$

For Kafka with `min.insync.replicas =  $m$` , the cluster also requires **at least m in-sync replicas** to accept writes. Thus:

$$\text{tolerated failures while still writable} = r - m.$$

4. Quorum Systems: $R + W > N$

4.1 The intersection argument

Leaderless systems (Cassandra, DynamoDB) use *read* and *write quorums* instead of a leader. Let:

- N = total number of replicas of a piece of data,
- W = number of replicas a write must acknowledge,
- R = number of replicas a read must consult.

Theorem – Strong Consistency Inequality

A read is guaranteed to see the most recent write if and only if

$$R + W > N.$$

Sketch. The write touches W replicas, the read touches R replicas. If $R + W > N$, then by the pigeonhole principle the two sets must share at least one common replica—which holds the latest version. Conversely, if $R + W \leq N$, the two sets can be disjoint, so the read may miss the latest write. $\square$

4.2 Tuning the trade-off

The choice of (R, W) controls the read/write latency trade-off:

Configuration	R	W	Trade-off
Read-optimised	1	N	Fast reads, slow writes, full consistency
Write-optimised	N	1	Fast writes, slow reads, full consistency
Balanced quorum	$\lceil N/2 \rceil + 1$	$\lceil N/2 \rceil + 1$	Symmetric latency, full consistency
Eventual	1	1	Lowest latency, no consistency guarantee

4.3 Connection to the CAP theorem

The quorum inequality is the formal counterpart of the CAP theorem. When the network partitions, you may not be able to reach W replicas synchronously. You then face a binary choice:

- Refuse the write $\Rightarrow$ **CP** (consistency over availability).
- Accept the write on fewer than W replicas $\Rightarrow$ **AP** (availability over consistency).

5. Throughput, Latency, and Little’s Law

5.1 Why Kafka measures both

A streaming pipeline is characterised by two main numbers:

- **Throughput** λ : events processed per second.
- **Latency** L : end-to-end time from production to consumption.

These are not independent: they are linked by a remarkable identity that holds for *any* stable queueing system, regardless of the arrival or service distributions.

Theorem – Little’s Law

For any stable system in steady state, the average number of items in the system $\overline{N}$, the average arrival rate λ , and the average time spent in the system L satisfy:

$$\overline{N} = \lambda \cdot L.$$

5.2 Application to Kafka

If your producers send $\lambda = 10,000$ events per second, and the end-to-end latency through Kafka is $L = 50$ ms, then on average:

$$\overline{N} = 10,000 \times 0.05 = 500 \text{ events “in flight”}.$$

This number drives capacity planning for partition count, broker memory, and consumer parallelism.

Parallelism and Throughput

For a Kafka topic with P partitions and a single consumer group, the maximum effective parallelism is $\min(P, C)$, where C is the number of consumers in the group. Therefore:

$$\lambda_{\max} = \min(P, C) \cdot \lambda_{\text{per-consumer}}.$$

Adding more consumers than partitions yields no speed-up—the extra consumers stay idle.

6. Exercises

The two exercises that follow combine the four building blocks above. They are self-contained: every numerical value needed is given in the statement.

Exercise 1 – Replication, Reliability, and Quorums

A team operates a Kafka cluster with $N = 5$ brokers. Each broker has an **independent** probability $p = 0.02$ of being unavailable during any given hour.

Part A. A topic is created with replication factor $r = 3$ and `min.insync.replicas = 2`.

- (a) Compute the probability that all 3 replicas of a partition are unavailable simultaneously (i.e. probability of data *loss* during one hour).
- (b) Compute the probability that the partition becomes *read-only*—that is, fewer than 2 replicas are available (writes blocked) but at least 1 is up (no loss). Provide the numerical value.
- (c) Compute the probability that the partition is *fully writable* (at least 2 of 3 replicas up).

Part B. The team also runs a Cassandra cluster with $N = 5$ replicas of every key. They want *strong consistency*, i.e. every read must see the latest write.

- (d) State the inequality that R and W must satisfy.
- (e) List **all** pairs (R, W) with $1 \leq R, W \leq 5$ that satisfy strong consistency.
- (f) Among those pairs, which one minimises the *maximum* of R and W (i.e. best worst-case latency)?

 Exercise 2 – Hash Partitioning, Throughput, and Little’s Law

An e-commerce company streams clickstream events into Kafka. The producer sends events with a key equal to the `user_id`. There are $P = 8$ partitions in the topic.

Part A. Suppose $n = 800,000$ user events are sent during one hour, with *uniformly distributed* `user_ids`. Model partition assignment as the balls-and-bins model with a perfect hash.

- (a) Compute the expected number of events per partition $\mathbb{E}[X_i]$ and the standard deviation $\sigma(X_i)$.
- (b) Compute the coefficient of variation $\sigma(X_i)/\mathbb{E}[X_i]$ and comment on whether the load is well-balanced.
- (c) Suppose now that 40% of all events come from a single “power user.” What is the load of the partition holding that user’s events (in expectation)? Compare with the average load. What is the practical consequence?

Part B. The consumer group has C consumers and the team measures an end-to-end latency of $L = 80$ ms per event. Each consumer can process **at most** 4,000 events per second.

- (d) The producer rate is $\lambda = 20,000$ events/second. What is the minimum number of consumers C needed to keep up with this rate, given $P = 8$ partitions?
- (e) Using Little’s Law, compute the average number of events “in flight” in the pipeline at steady state.
- (f) The team decides to increase the producer rate to $\lambda' = 40,000$ events/second. They double the number of consumers from 5 to 10, but observe *no improvement* in throughput. Explain mathematically.

7. Summary

Key Formulas to Remember

Concept	Formula
Hash partitioning load	$X_i \sim \mathcal{B}(n, 1/P), \quad \mathbb{E}[X_i] = n/P, \quad \text{Var}(X_i) = (n/P)(1 - 1/P)$
Probability of data loss	$\mathbb{P}(\text{loss}) = p^r$ (independent failures, replication factor r)
Tolerated simultaneous failures	$r - 1$ (with replication factor r)
Strong consistency (quorum)	$R + W > N$
Little's Law	$\bar{N} = \lambda \cdot L$
Kafka effective parallelism	$\lambda_{\max} = \min(P, C) \cdot \lambda_{\text{per-consumer}}$

7.1 Going further

The mathematical themes of this document are studied in much greater depth in:

- Mitzenmacher & Upfal, *Probability and Computing* (balls-and-bins, Chernoff bounds).
- Kleppmann, *Designing Data-Intensive Applications*, Ch. 5–9 (replication, partitioning, consistency).
- Lynch, *Distributed Algorithms* (consensus, FLP impossibility).